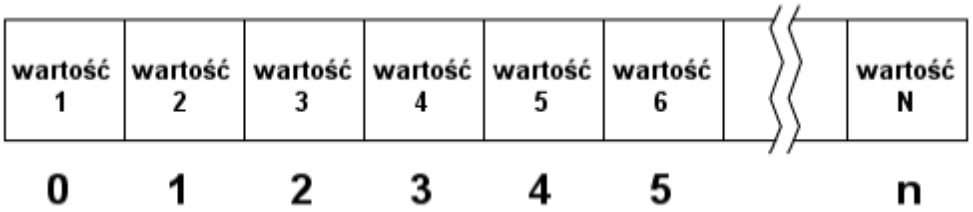
Tablice w PHP - 2019

Tablice są to struktury danych pozwalające na przechowywanie uporządkowanego zbioru elementów. Najprostszą tablicę można sobie wyobrazić jako wektor elementów.

Poszczególne pola danych nazywamy wtedy komórkami, a każda z nich ma identyfikujący ją indeks (mówimy wtedy o tablicach indeksowanych numerycznie) i może zawierać pewną wartość. Pierwsza komórka (zawierająca *wartość 1*) ma indeks 0, druga — indeks 1, trzecia — indeks 2 itd.



W celu utworzenia tablicy używamy słowa kluczowego array.
Taka konstrukcja ma schematyczną postać:

$*tablica* = array(*wartość1*, *wartość2*,..., *wartośćN*);

$*tablica* to zmienna typu tablicowego, dzięki której będziemy mogli się do tej struktury odwoływać, natomiast *wartość1*, *wartość2* itd. to wartości znajdujące się w kolejnych komórkach.

Można również zastosować zapis w postaci:

$*tablica* = array(
wartość1,
wartość2,
...,
wartośćN);

```
<?php
$kolory = array("czerwony", "zielony", "niebieski");
echo "kolory[0] = ", $kolory[0], "<br />";
echo "kolory[1] = ", $kolory[1], "<br />";
echo "kolory[2] = ", $kolory[2], "<br />";
?>
```

Aby uzyskać dostęp do wartości zapisanej w danej komórce, należy podać jej numer (indeks) w nawiasie kwadratowym występującym za nazwą tablicy. Należy przy tym pamiętać, że indeksowanie tablicy zaczyna się od 0, co oznacza, że indeksem pierwszej komórki jest 0, a NIE 1.

Pierwszej $kolory[0], drugiej — $kolory[1], a trzeciej — $kolory[2].
Jeśli tablica ma duży rozmiar, do odczytu jej zawartości lepiej użyć pętli. Może być to zarówno zwykła pętla for, jak i pętla typu foreach.

```
<?php
$kolory = array("czerwony", "zielony", "niebieski");
for($i = 0; $i < 3; $i++){
echo $kolory[$i], "<br />";
}
echo "<br />";
foreach($kolory as $kolor){
echo $kolor, "<br />";
}
?>
```

Tworzenie tablic z użyciem funkcji **array**.

Opis 1
definiowanie tablicy jednowymiarowej:
$t=array("Iwa", "Kasia ", "Basia ");
tablica zawiera trzy imiona o indeksach 0 1 2

Opis 2
definiowanie tablicy gdzie **indeksami** są *imiona* a **wartościami** *nazwiska*:
$t=array("Iwa"=>"Kowalska", "Kasia "=>"Nowak", "Basia "=>"Kwiatkowska");

funkcja count

funkcja **count($nazwa_tablicy)**. Zwraca ona ilość elementów w tablicy podanej jako parametr.

Pętla foreach

Pętla foreach pozwala na dostęp do kolejnych elementów lub właściwości.

Może ona występować w dwóch postaciach:

foreach($*tablica* as $*wartość*){
instrukcje;
}

Oraz

```
foreach($tablica as $klucz => $wartość){
instrukcje;
}
```

Różnica pomiędzy obiema postaciami jest taka, że w drugim przypadku otrzymujemy dodatkowo wartość aktualnego klucza. Jeśli te wartości nie są potrzebne, można skorzystać z pierwszej formy pętli.

Zastosowanie pierwszej wersji pętli foreach

```
<?php
$tab = array(
1 => 'jeden',
2 => 'dwa',
'trzy' => 3,
'cztery' => 4
);
foreach($tab as $v){
echo "$v <br />";
}
?>
```

Tablica $tab zawiera cztery komórki o indeksach (kluczach) 1, 2, trzy oraz cztery, w których zapisane są ciągi znaków oraz wartości liczbowe. Wewnątrz pętli pod zmienną $v są podstawiane wartości kolejnych komórek, stąd na ekranie pojawią się wartości: jeden, dwa, 3, 4.

Jeśli chcemy uzyskać dodatkowo nazwy indeksów, musimy skorzystać z drugiej z przedstawionych konstrukcji. W każdym przebiegu pętli pod zmienną $key jest podstawiana nazwa aktualnego klucza, natomiast pod zmienną $v — jego wartość.

Wykorzystanie drugiej wersji pętli foreach

```
<?php
$tab = array(
1 => 'jeden',
2 => 'dwa',
'trzy' => 3,
'cztery' => 4
);
foreach($tab as $key => $v){
echo "tab[$key] = $v <br />";
};
?>
```

Instrukcje break i continue

Instrukcje break i continua pozwalają na modyfikacje zachowań pętli i można je stosować z każdym ich rodzajem, niezależnie od tego, czy będzie to for, while, do…While czy foreach. Instrukcja break powoduje przerwanie działania pętli.

Wykorzystanie instrukcji break do przerwania pętli

```
<?php
$i = 0;
while(true){
echo "napis [i = $i] <br />";
if($i >= 9) break;
$i++;
}
?>
```

O ile instrukcja break powodowała przerwanie wykonywania pętli oraz jej opuszczenie, o tyle continue powoduje przejście do jej kolejnej iteracji. Jeśli zatem wewnątrz pętli znajdzie się instrukcja continue, bieżąca iteracja (przebieg) zostanie przerwana oraz rozpocznie się kolejna (chyba że bieżąca była ostatnią).

Warto wiedzieć, że ze względu na specyficzną budowę tablic w PHP wartości w poszczególnych komórkach tablicy nie muszą być tego samego typu (w odróżnieniu od wielu innych języków programowania).

Prawidłowa będzie zatem konstrukcja w postaci:

```
$tab = array("czerwony", 10, 1.14);
```

W takim przypadku tablica $tab będzie zawierała trzy dane: łańcuch znaków, wartość całkowitą oraz wartość rzeczywistą. Nie ma również konieczności, aby wypełniać tablice danymi już podczas ich tworzenia. Mogą być one także przypisywane w dalszej części skryptu. Zamiast na przykład pisać:

```
$kolory = array("czerwony", "zielony", "niebieski");
```

można wykorzystać serię instrukcji w postaci:

```
$kolory = array();   //pusta tablica
$kolory[0] = "czerwony";
$kolory[1] = "zielony";
$kolory[2] = "niebieski";
```

W ten sam sposób można również zmieniać zawartość poszczególnych komórek.

Zmiana zawartości komórek tablicy

```
<?php
$kolory = array();
$kolory[0] = "czerwony";
$kolory[1] = "zielony";
$kolory[2] = "niebieski";
echo("Pierwotna zawartość tablicy: <br />");
for($i = 0; $i < 3; $i++){
$kolor = $kolory[$i];
echo "kolory[$i] = $kolor <br />";
}
$kolory[0] = "żółty";
$kolory[1] = "fioletowy";
$kolory[2] = "czarny";
echo("<br />Zawartość tablicy po zmianach: <br />");
for($i = 0; $i < 3; $i++){
$kolor = $kolory[$i];
echo "kolory[$i] = $kolor <br />";
}
?>
```

Wynik wywołania:

Pierwotna zawartość tablicy:
kolory[0] = czerwony
kolory[1] = zielony
kolory[2] = niebieski

Zawartość tablicy po zmianach:
kolory[0] = żółty
kolory[1] = fioletowy
kolory[2] = czarny

Wielkość tablicy nie jest z góry ustalona i rośnie wraz z dodawaniem kolejnych elementów. To bardzo wygodna cecha języka. Otóż po napotkaniu instrukcji w postaci:

$*tablica*[$*indeks*] = *wartość*;

PHP sprawdza, czy istnieje tablica $*tablica* oraz czy jest w niej indeks $*indeks*. Możliwe są trzy sytuacje:

1. Jeśli oba warunki są spełnione, elementowi wskazywanemu przez $*indeks* jest przypisywana *wartość*. Poprzednia zawartość tego elementu jest tracona.
2. Jeśli nie ma indeksu $*indeks*, jest on najpierw tworzony, a dopiero potem przypisuje się mu $*wartość*.
3. Jeśli nie ma samej tablicy, zostanie ona utworzona, a następnie wykonane będą czynności z punktu 2.

Nie ma konieczności przypisywania indeksów w uporządkowanej kolejności. Mogą istnieć w tablicy tylko wybrane indeksy, np.: 2, 5 i 8. Z powyższego opisu wynika również, że nie ma formalnej konieczności tworzenia tablicy za pomocą słowa array. Jeżeli w skrypcie zostanie napotkana instrukcja przypisująca dowolnemu indeksowi tablicy jakąś wartość, ale tablica o danej nazwie nie będzie istniała, zostanie ona niejawnie utworzona.

Indeks można też podawać ze zmiennej, z innej tablicy czy funkcji – z dowolnego wyrażenia zwracającego wartość.

```
<?php
$tab1[] = 1;
$tab1[] = 0;
$tab1[] = 3;
$tab1[] = 2;
$tab2[] = "Pierwszy";
$tab2[] = "Drugi";
$tab2[] = "Trzeci";
$tab2[] = "Czwarty";
echo $tab2[$tab1[2]];
echo $tab2[$tab1[2]];
?>
```

Wykonanie: czwarty

Oprócz tablic indeksowanych numerycznie w PHP istnieją również tablice asocjacyjne. W tablicach tego typu każdemu indeksowi można nadać unikalną nazwę, czyli zamiast indeksów 0, 1, 2 itd. mogą występować indeksy: kolor, autor, procesor itp.

Najczęściej też zamiast terminu „indeks” stosuje się określenie „klucz”. W tablicy asocjacyjnej występują pary klucz – wartość, w których każdy klucz jednoznacznie identyfikuje przypisaną mu wartość. Kluczami mogą być ciągi znaków bądź wartości całkowite. Tablicę tego typu tworzy się, podobnie jak w przypadku tablic klasycznych indeksowanych numerycznie, za pomocą słowa kluczowego array, konstrukcja ta ma jednak nieco inną postać. Schematycznie wygląda to następująco:

```
array(
klucz1 =>wartość1,
klucz2 =>wartość2,
...
kluczN=>wartośćN
);
```

Tworzenie tablicy asocjacyjnej

```
<?php
$kolory = array(
"kolor1" => "czerwony",
"kolor2" => "zielony",
"kolor3" => "niebieski"
);
echo "Zawartość tablicy:<br />";
echo "kolory['kolor1'] = ", $kolory['kolor1'], "<br />";
echo "kolory['kolor2'] = ", $kolory['kolor2'], "<br />";
echo "kolory['kolor3'] = ", $kolory['kolor3'], "<br />";
?>
```

Drugim ze sposobów tworzenia tablicy asocjacyjnej jest użycie składni z nawiasami kwadratowymi, podobnie jak miało to miejsce w przypadku tablic indeksowanych numerycznie.

Schematycznie taka konstrukcja ma postać:

```
$nazwa_tablicy['nazwa_klucza'] = wartość_klucza;
```

```
$kolory['kolor1'] = "czerwony";
$kolory['kolor2'] = "zielony";
$kolory['kolor3'] = "niebieski";
```

```
<?php
$owoce = array ("d"=>"mango", "a"=>"papaja", "b"=>"banan", "c"=>"aronia");
var_dump( $owoce );
?>
```

Do odczytu tablic asocjacyjnych można użyć pętli. Nie może być to jednak pętla typu for, gdyż nie jest ona w stanie stwierdzić, jakie są wartości kluczy. Tablice asocjacyjne najczęściej są obsługiwane przez pętle typu foreach. Taka pętla potrafi pobrać kolejne wartości kluczy;

```
<?php
$kolory = array(
"kolor1" => "czerwony",
"kolor2" => "zielony",
"kolor3" => "niebieski"
);
echo "Zawartość tablicy:<br />";
foreach($kolory as $kolor){ echo $kolor, "<br />"};
?>
```

Modyfikacji zawartości tablic asocjacyjnych dokonuje się w sposób analogiczny jak w przypadku tablic klasycznych. Oczywiście zamiast indeksów numerycznych trzeba zastosować wartości kluczy. Aby zatem przypisać nową wartość już istniejącemu kluczowi, trzeba skorzystać z konstrukcji, której schematyczna postać jest następująca:

$*tablica*['*klucz*'] = *wartość*;

Jeśli więc istnieje tablica $kolory, a w niej klucz kolor1, to wykonanie instrukcji:

```
$kolory['kolor1'] = 'zielony';
```

spowoduje zmianę jego wartości na ciąg znaków zielony.

Przykłady

array(3) { [0]=> string(5) "Litwo" [1]=> string(14) " ojczyzno moja" [2]=> string(24) " Ty jesteś jak zdrowie." }

Array ([0] =>Litwo,ojczyznomoja,Ty jesteś jak zdrowie.)

1Array ([0] => Litwo, [1] => ojczyzno [2] => moja, [3] => Ty [4] => jesteś [5] => jak [6] => zdrowie.)

array(7) { [0]=> string(6) "Litwo," [1]=> string(8) "ojczyzno" [2]=> string(5) "moja," [3]=> string(2) "Ty" [4]=> string(7) "jesteś" [5]=> string(3) "jak" [6]=> string(8) "zdrowie." }

```
<?php

$tekst = "Litwo, ojczyzno moja, Ty jesteś jak zdrowie.";
$wyrazy = explode(",", $tekst);
var_dump($wyrazy);echo("<br>");echo("<br>");
$tekst = "Litwo,ojczyznomoja,Ty jesteś jak zdrowie.";
$wyrazy = explode(";", $tekst);
print_r($wyrazy);echo("<br>");echo("<br>");

echo count($wyrazy);
$tekst = "Litwo, ojczyzno moja, Ty jesteś jak zdrowie.";

$wyrazy = explode(" ", $tekst);
print_r($wyrazy);echo("<br>");echo("<br>");
// wyświetli ilość wyrazów w zmiennej $tekst
var_dump($wyrazy);echo("<br>");echo("<br>");
echo count($wyrazy);

?>
```

Tablice są bardzo specyficznym typem zmiennych – są to, najprościej mówiąc, zmienne zawierające w sobie uporządkowany zbiór zmiennych. Do zmiennych tych uzyskuje się dostęp przez liczbę w nawiasie kwadratowym podane bezpośrednio po nazwie zmiennej – tablicy. Liczba ta to tak zwany indeks – numer kolejny zmiennej w tablicy. Tak samo przypisuje się wartość do tablicy.

Przykład 4.1. Tworzenie tablicy

```
<?php
$tablica[0] = "Wpis numer 0";
$tablice[1] = "Wpis numer 1";
$tablica[2] = "Wpis numer 2";
echo $tablica[2]; // Wyświetlony zostanie napis "Wpis numer 2";
?>
```

Aby po prostu dodać kolejny wpis na końcu tabeli wystarczy przy przypisywaniu wartości nie wpisywać indeksu do nawiasów kwadratowych. Jeśli w ten sposób dodawane są wpisy do nowej tablicy, to pierwszy wpis ma indeks 0.
Indeks można też podawać ze zmiennej, z innej tablicy czy funkcji – z dowolnego wyrażenia zwracającego wartość.

Przykład 4.2. Indeksy tablic

```
<?php
$tab1[] = 1;
```

```
$tab1[] = 0;
$tab1[] = 3;
$tab1[] = 2;
$tab2[] = "Pierwszy";
$tab2[] = "Drugi";
$tab2[] = "Trzeci";
$tab2[] = "Czwarty";
echo $tab2[$tab1[2]];
?>
```

Elementem tablicy może być każdy typ zmiennej (z innymi tablicami i obiektami włącznie).
Innym ze sposobów deklaracji tablic jest użycie słowa kluczowego Array:

```
<?php
$owoce = array ("mango",  "papaja", "banan", "aronia");
var_dump( $owoce );
?>
```

Wynik:
array(4) { [0]=> string(5) "mango" [1]=> string(6) "papaja" [2]=> string(5) "banan" [3]=> string(6) "aronia" }

Przykład 4.4.

```
<?php
$owoce = array ("d"=>"mango", "a"=>"papaja", "b"=>"banan", "c"=>"aronia");
var_dump( $owoce );
?>
```

Wynik: array(4) { ["d"]=> string(5) "mango" ["a"]=> string(6) "papaja" ["b"]=> string(5) "banan" ["c"]=> string(6) "aronia" }

Przykład 4.5.

```
<?php
$imiona = Array('Marcin', 100 => 'Daniel', 200 => 'Magda', 'Paulina');
echo $imiona[0]; // wyświetli Marcin
echo $imiona[200]; // wyświetli Magda
$imiona[201] = 'Katarzyna'; // zmiana wartości z Paulina na Katarzyna
echo $imiona[201]; // wyświetli Katarzyna
?>
```

4.2 Tablica asocjacyjna

W PHP występuje też inny rodzaj tablic, tak zwane tablice asocjacyjne (zwane też czasem haszami – hash table). Są to tablice, w których zamiast indeksów liczbowych używa się identyfikatorów znakowych (kluczy):

Przykład 4.6. Tablice asocjacyjne

```
<?php
$tablica["imie"] = "Jan";
$tablica["nazwisko"] = "Kowalski";
$tablica["adres"] = "Polna 1";
echo $tablica["imie"]." ".$tablica["nazwisko"]., ul. ".$tablica["adres"]."\n";
?>
```

Przykład 4.7. Tablice asocjacyjne

```
<?php
// tworzymy tablicę asocjacyjną
$tablica = array("imie" => "Jan", "nazwisko" => "Kowalski", "email" => "jankowal@gmail.com");
// Wyświetlamy
foreach ($tablica as $klucz => $dana)
{
echo 'Klucz to <b>'.$klucz.'</b> a jego wartość to <b>'.$dana.'</b>';
}
?>
```

4.3 Tablice wielowymiarowe

PHP umożliwia także deklaracje tzw. tablic wielowymiarowych. Polega to na tworzeniu kolejnych węzłów elementu dając przy tym wrażenie drzewa elementów. *Tablice* wielowymiarowe można deklarować podając kolejne indeksy w nawiasach kwadratowych:

```
<?
$dane[0]['imię'] = 'Jan';
$dane[0]['nazwisko'] = 'Kowalski';
$dane[0]['ulica'] = 'Kowalowska';
$dane[1]['imię'] = 'Maciej';
$dane[1]['nazwisko'] = 'Nowak';
$dane[1]['ulica'] = 'Nowakowska';
print_r($dane);
?>
```

4.4 Tworzenie ciągów z tablic i odwrotnie

PHP umożliwia zamianę ciągów na tablice i odwrotnie. Zamiana ciągu na tablicę jest bardzo przydatna jeśli zachodzi potrzeba wyciągnięcie jakiegoś fragmentu danych z ciągu. Załóżmy że w odczytaliśmy z pliku z danymi (o odczycie z plików w jednym z kolejnych rozdziałów) linię z logu zapisanego przez licznik WWW: “12/11/2000;19:23:33;Netscape Navigator;192.168.1.1″. Jak widać dane rozdzielone są średnikami. Do rozdzielania ciągów na tablicę służy funkcja explode(). Jako pierwszy parametr trzeba do niej podać znak lub dłuższy ciąg który oddziela kolejne pola, jako drugi ciąg do rozdzielenia. Opcjonalnie można podać trzeci argument, który oznacza maksymalną liczbę pól – jeśli jest ich więcej niż ta liczba, to ostatnie pole będzie zawierało wszystkie pozostałe pola. Funkcja zwraca tablicę zawierającą kolejne pola.

Przykład 4.8.

```
<?php
$dane = "12/11/2000;19:23:33;Netscape Navigator;192.168.1.1";
$tablica = explode(";", $dane);
print_r($tablica);
?>
```

Array ([0] => 12/11/2000 [1] => 19:23:33 [2] => Netscape Navigator [3] => 192.168.1.1)

Jest także rozszerzona wersja funkcji explode: split(). Różni się ona tym, że zamiast prostego ciągu znaków rozdzielających pola, akceptuje ona wyrażenia regularne.
Czasem potrzebne jest działanie w drugą stronę: złącznie pól tablicy w jeden ciąg, w którym pola oddzielone są jakimś znakiem (lub kilkoma). Do tego służy funkcja implode(). Jako pierwszy parametr podawany jest ciąg za pomocą którego “sklejane” są elementy tablicy, a jako drugi właśnie tablica do posklejania. Zwracany jest ciąg zawierający posklejane elementy. Jako przykład zastosowania może posłużyć właśnie zapisywanie danych o użytkowniku w aplikacji licznika odwiedzin – tablica zawiera dane o odwiedzającym, a potrzebny jest ciąg pooddzielany średnikami.

Przykład 4.9.

```
<?php
$dane = implode(";", $tablica);
?>
```

Funkcje tablic - dodawanie nowych elementów

array_push

Wstawia jeden lub więcej elementów na koniec tablicy

Opis

int **array_push** (array &tablica, mixed wartosc [, mixed ...])

array_push() traktuje zmienną *tablica* jako stos i wstawia przekazane parametry na koniec podanej tablicy. Długość parametru *tablica* zwiększa się o liczbę przekazanych wartości.

Przykład 4.10

```
<?php
$stos = array("pomarańcza", "banan");
array_push($stos, "jabłko", "malina");
print_r($stos);
?>
```

Po wykonaniu powyższego przykładu zmienna *$stos* będzie miała następujące elementy:

```
Array
(
   [0] => pomarańcza
   [1] => banan
   [2] => jabłko
   [3] => malina
)
```

array_unshift

Wstawia jeden lub więcej elementów na początek tablicy

Opis

int array_unshift (array &tablica, mixed wartość [, mixed ...])

array_unshift() wstawia jeden lub więcej przekazanych jako parametry elementów na początek tablicy *tablica*. Zauważ, że lista elementów wstawiana jako całość, więc elementy zostają w takim samym porządku. Wszystkie klucze liczbowe zostaną zmodyfikowane tak, aby ich wartości zaczynały się od zera, podczas gdy klucze znakowe nie zostaną zmienione.

Funkcja zwraca nową liczbę elementów w tablicy *tablica*.

Przykład 4.11

```
<?php
$kolejka = array ("pomarańcza", "banan");
array_unshift ($kolejka, "jabłko", "malina");
?>
```

Zmienna *$kolejka* będzie miała następujące elementy:

```
Array
(
   [0] => jabłko
   [1] => malina
   [2] => pomarańcza
   [3] => banan
)
```

Funkcje tablic - usuwanie elementów

array_pop

Zdejmuje element z końca tablicy

Opis

mixed **array_pop** (array &tablica)

array_pop() zdejmuje i zwraca ostatnią wartość tablicy *tablica*, skracając tą tablicę o jeden element. Jeśli *tablica* jest pusta (lub nie jest tablicą), zwracana jest wartość **NULL**.

Przykład 4.12

```
<?php
$stos = array("pomarańcza", "banan", "jabłko", "malina");
$owoc = array_pop($stos);
print_r($stos);
?>
```

Po wykonaniu powyższego kodu *$stos* będzie miał tylko trzy elementy:

```
Array
(
    [0] => pomarańcza
    [1] => banan
    [2] => jabłko
)
```

a *malina* będzie przypisana do zmiennej *$owoc*.

array_shift

Usuwa element z początku tablicy

Opis

mixed **array_shift** (array &tablica)

array_shift() usuwa pierwszą wartość parametru *tablica* i zwraca go skracając tą tablicę o jeden element przesuwając wszystkie pozostałe elementy w dół. Wszystkie klucze liczbowe zostaną zmodyfikowane tak, aby ich wartości zaczynały się od zera, podczas gdy klucze znakowe nie zostaną zmienione. Jeśli *tablica* jest pusta (lub nie jest tablicą), zwracana jest wartość **NULL**.

Notatka: Ta funkcja **zresetuje()** wskaźnik tablicy po swoim wykonaniu.

Przykład 4.13

```
<?php
$stos = array ("pomarańcza", "banan", "jabłko", "malina");
$owoc = array_shift ($stos);
?>
```

Zmienna *$stos* będzie miała 3 elementy:

```
Array
(
    [0] => banan
    [1] => jabłko
    [2] => malina)
```

a *pomarańcza* będzie przypisana do zmiennej *$owoc*

Funkcja **unset();**
pozwala usunąć pojedynczą, lub więcej zmiennych, lub element tablicy.

```
<?php
unset($owoce[1]); // usunięcie pojedynczej zmiennej
unset($owoce[o], $owoce[3], $owoce[5]); // usunięcie wielu zmiennych
?>
```

Funkcje tablic - losowanie

rand

Generuje losową liczbę stałoprzecinkową

int **rand** ([int $min [, int $max]])

Jeśli wywołana bez opcjonalnych argumentów *min* i *max*, funkcja **rand()** zwraca pseudolosową liczbę stałoprzecinkową z przedziału pomiędzy 0 a **RAND_MAX**. Dla uzyskania liczby losowej z przedziału np. od 5 do 15 (włącznie), należy wywołać *rand (5,15)*.

Przykład 4.14

```
<?php
echo rand() . "\n";
echo rand() . "\n";

echo rand(5, 15);
?>
```

Powyższy przykład wyświetli coś podobnego do:

```
7771
22264
11
```

Sortowanie tablic

PHP ofertuje cały zestaw funkcji służących do sortowania tablic. Są to:

sort() - sortuje zwykłe tablice (nie asocjacyjne) w kolejności alfabetycznej

rsort() - sortuje zwykłe tablice (nie asocjacyjne) w odwróconej kolejności

asort() - sortuje tablice asocjacyjne zachowując przypisanie kluczy do wartości

arsort() - sortuje w odwrotnej kolejności tablice asocjacyjne zachowując przypisanie kluczy do wartości.

ksort() - sortuje tablice asocjacyjne według kluczy

Przykład 4.15

```
<?php

$owoce = array ("d"=>"mango", "a"=>"papaja", "b"=>"banan", "c"=>"aronia");

asort ($owoce);

echo "<pre>";

print_r ($owoce);

echo "</pre>";

?>
```

Powyższy przykład wyświetli coś podobnego do:

```
Array
(
    [c] => aronia
    [b] => banan
    [d] => mango
    [a] => papaja
)
```

Spacer po tablicy.

next

Przesuń do przodu wewnętrzny wskaźnik tablicy

Przesuwa wewnętrzny wskaźnik tablicy i jedną pozycję do przodu i zwraca element tablicy aktualnie wskazywany przez wskaźnik, lub **FALSE** jeśli nie ma już więcej elementów.

current

Zwraca bieżący element tablicy

Funkcja **current()** po prostu zwraca element tablicy, na który aktualnie wskazuje wewnętrzny wskaźnik. Nie przesuwa ona wskaźnika. Jeśli wewnętrzny wskaźnik jest poza końcem listy elementów, **current()** zwraca **FALSE**.

prev

Cofnij wewnętrzny wskaźnik tablicy

Zwraca wartość z tablicy z miejsca poprzedniego od tego na które wskazywał wewnętrzny wskaźnik pliku, lub **FALSE** jeśli nie ma już więcej elementów.

reset

Ustaw wewnętrzny wskaźnik tablicy na jej pierwszy element

reset() przewija wewnętrzny wskaźnik tablicy parametru *tablica* na jego pierwszy element i zwraca jego wartość, lub **FALSE** jeśli tablica jest pusta.

Przykład 4.16

```
<?php

$tablica = array('krok pierwszy', 'krok drugi', 'krok trzeci', 'krok czwarty');

echo current($tablica) . "<br />\n"; // "krok pierwszy"

next($tablica);

next($tablica);

prev($tablica);

echo current($tablica) . "<br />\n"; // "krok drugi"

reset($tablica);

echo current($tablica) . "<br />\n"; // "krok pierwszy

?>
```

Przeszukiwanie tablicy.

in_array

Sprawdza czy wartość istnieje w tablicy

bool **in_array** (mixed $igła , array $stóg_siana [, bool $ścisłe])

Przeszukuje *stóg_siana* w poszukiwaniu parametru *igła* i zwraca **TRUE** jeśli wartość została znaleziona lub **FALSE** w przeciwnym przypadku.

Przykład 4.17

```
<?php
$a = array(0,1,2,3,4,5);
```

```
in_array("0", $a) ;//return
true in_array("3", $a)
;//return true in_array("8",
$a);// return false
?>
```

array_search

Przeszukuje tablicę pod kątem podanej wartości i w przypadku sukcesu zwraca odpowiedni klucz

mixed **array_search** (mixed $igła , array $stóg_siana [, bool $ścisły])

Przeszukuje *stóg_siana* w poszukiwaniu parametru *igła* i zwraca odpowiedni klucz jeśli został on znaleziony lub **FALSE** w przeciwnym przypadku.

Przykład 4.18

```
<?php
$tablica = array(0 => 'niebieski', 1 => 'czerwony', 2 => 'zielony', 3 => 'czerwony');

$klucz = array_search('zielony', $tablica); // $klucz = 2;
$klucz = array_search('czerwony', $tablica); // $klucz = 1;
?>
```

4.10 Transformacje tablic.

array_intersect

Zwraca przecięcie tablic

array **array_intersect** (array $tablica1 , array $tablica2 [, array $ ...])

array_intersect() zwraca tablicę zawierającą wszystkie wartości tablicy *tablica1* które istnieją we wszystkich argumentach. Zauważ, że zachowywane są przypisania kluczy.

Przykład 4.19

```
<?php
$tablica1 = array ("a" => "zielony", "czerwony", "niebieski");
$tablica2 = array ("b" => "zielony", "żółty", "czerwony");
$wynik = array_intersect ($tablica1, $tablica2);
print_r($wynik);
?>
```

Powyższy przykład wyświetli:

```
Array
(
    [a] => zielony
    [0] => czerwony
)
```

array_diff

Zwraca różnice pomiędzy tablicami

array **array_diff** (array $tablica1 , array $tablica2 [, array $ ...])

array_diff() zwraca tablicę zawierającą wszystkie wartości tablicy *tablica1* które nie są obecne w innych tablicach-argumentach. Zauważ, że zachowywane są klucze.

Przykład 4.20

```
<?php
$tablica1 = array ("a" => "zielony", "czerwony", "niebieski", "czerwony");
$tablica2 = array ("b" => "zielony", "żółty", "czerwony");
$wynik = array_diff ($tablica1, $tablica2);

print_r($wynik);
?>
```

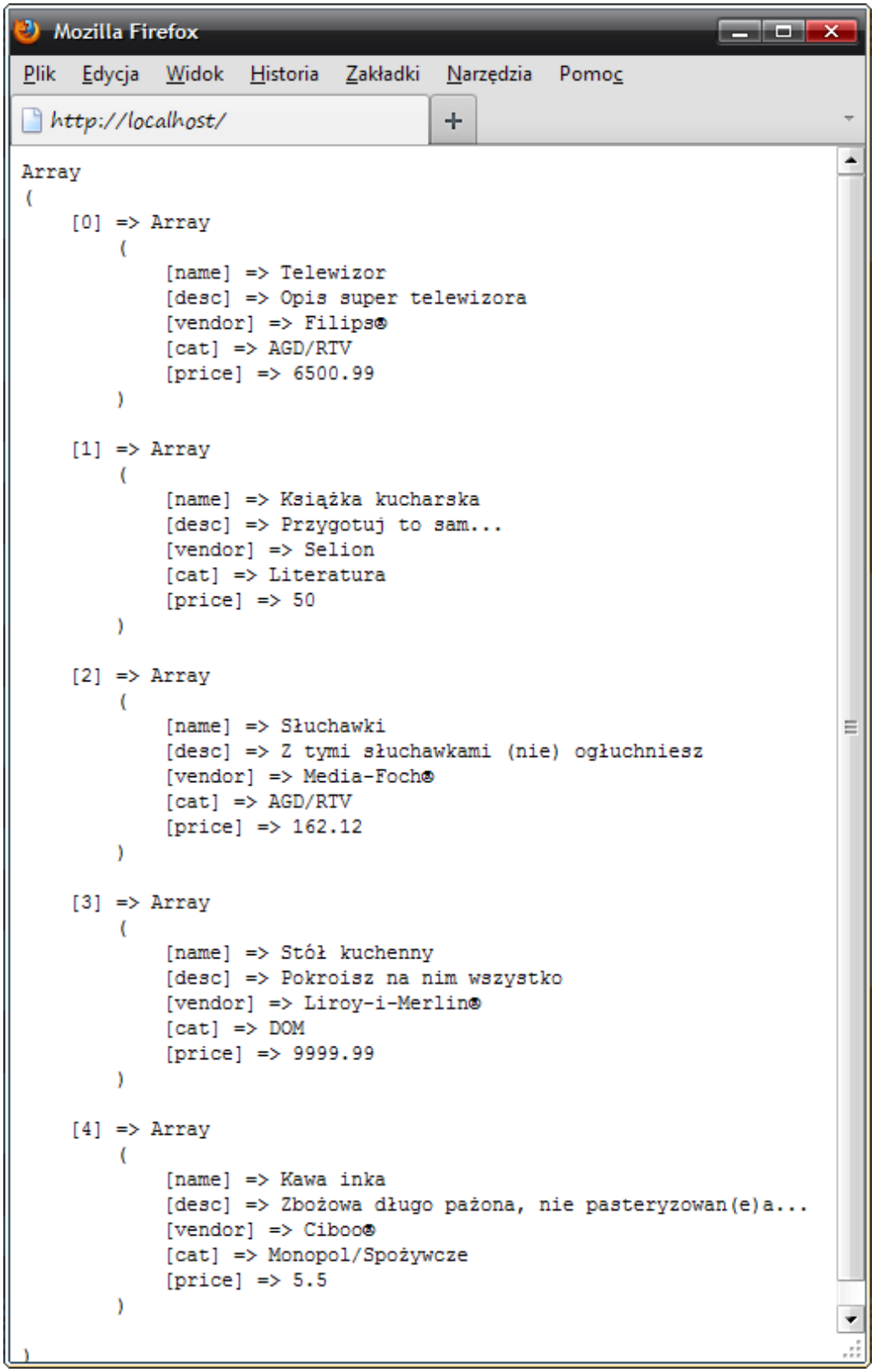
Wielokrotne wystąpienia w $tablica1 są traktowane w ten sam sposób. Powyższy przykład wyświetli:

```
Array
(
    [1] => niebieski
)
```

Ćwiczenie 4.1

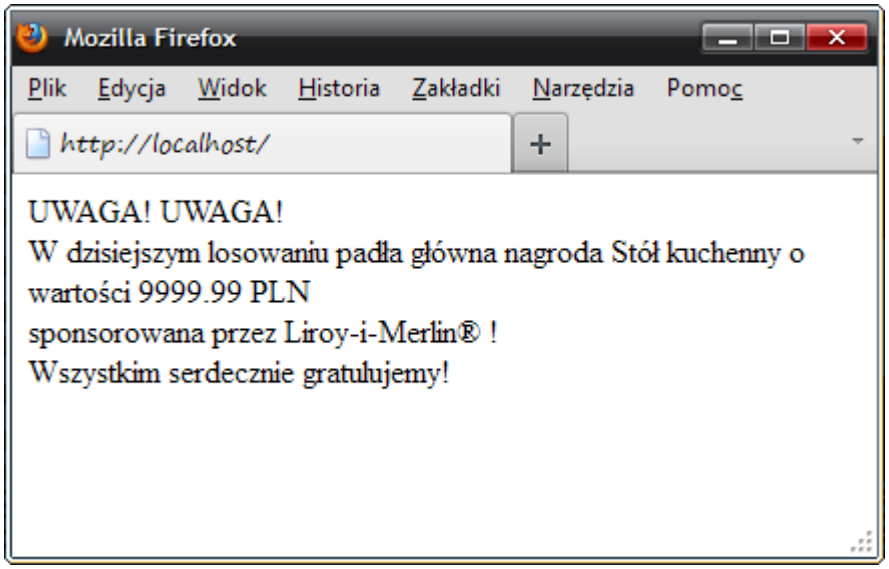
```
Przed dodaniem nowych produktów:

Array
(
    [0] => Telewizor
    [1] => Książka kucharska
    [2] => Słuchawki
    [3] => Stół kuchenny
    [4] => Kawa inka
)
---------------------------------------------------
Po modyfikacji (dodanie nowych elementów):

Array
(
    [0] => Telewizor
    [1] => Książka kucharska
    [2] => Słuchawki
    [3] => Stół kuchenny
    [4] => Kawa inka
    [5] => Telefon
    [6] => Plecak
)
---------------------------------------------------
Po modyfikacji (usunięcie elementów):

Array
(
    [0] => Telewizor
    [2] => Słuchawki
    [4] => Kawa inka
    [5] => Telefon
    [6] => Plecak
)
---------------------------------------------------
Dynamiczne tworzenie nowej tablicy:

To jest@przykładowy@ciąg znakówArray
(
    [0] => To jest
    [1] => przykładowy
    [2] => ciąg znaków
)
---------------------------------------------------
Telewizor[#]Słuchawki[#]Kawa inka[#]Telefon[#]Plecak
```

Ćwiczenie 4.2

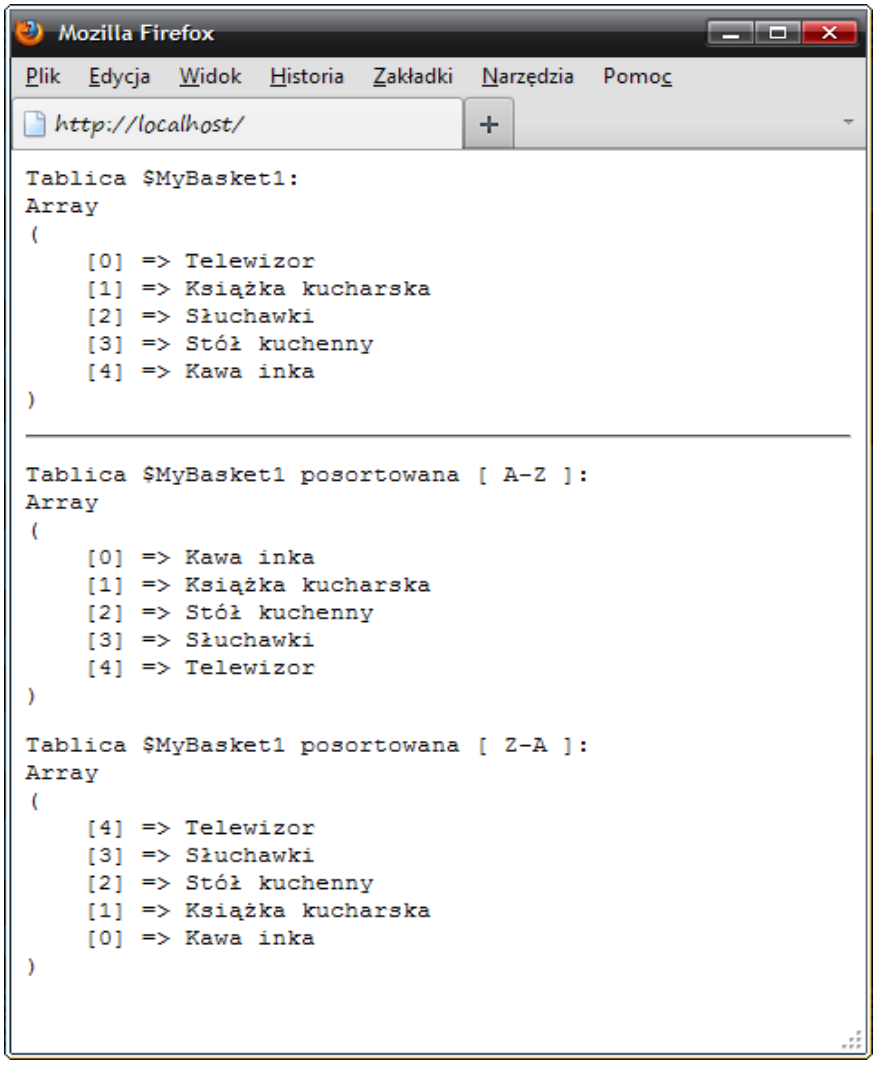



```
Array
(
    [0] => Array
        (
            [name] => Telewizor
            [desc] => Opis super telewizora
            [vendor] => Filips®
            [cat] => AGD/RTV
            [price] => 6500.99
        )

    [1] => Array
        (
            [name] => Książka kucharska
            [desc] => Przygotuj to sam...
            [vendor] => Selion
            [cat] => Literatura
            [price] => 50
        )

    [2] => Array
        (
            [name] => Słuchawki
            [desc] => Z tymi słuchawkami (nie) ogłuchniesz
            [vendor] => Media-Foch®
            [cat] => AGD/RTV
            [price] => 162.12
        )

    [3] => Array
        (
            [name] => Stół kuchenny
            [desc] => Pokroisz na nim wszystko
            [vendor] => Liroy-i-Merlin®
            [cat] => DOM
            [price] => 9999.99
        )

    [4] => Array
        (
            [name] => Kawa inka
            [desc] => Zbożowa długo pażona, nie pasteryzowan(e)a...
            [vendor] => Ciboo®
            [cat] => Monopol/Spożywcze
            [price] => 5.5
        )

)
```

Ćwiczenie 4.3



Ćwiczenie 4.4



Ćwiczenie 4.5

Mozilla Firefox

Plik Edycja Widok Historia Zakładki Narzędzia Pomoc

http://localhost/

```
Tablica $MyBasket1:
Array
(
    [0] => Telewizor
    [1] => Książka kucharska
    [2] => Słuchawki
    [3] => Stół kuchenny
    [4] => Kawa inka
)
```

Wewnętrzny wskaźnik tablicy znajduje się nad elementem Telewizor

Przesuwamy wskaźnik tablicy o jeden w prawo...

Przesuwamy wskaźnik tablicy o jeden w prawo...

Wewnętrzny wskaźnik tablicy znajduje się nad elementem Słuchawki

Wskaźnik tablicy został zresetowany...

...i teraz znajduje się nad elementem Telewizor

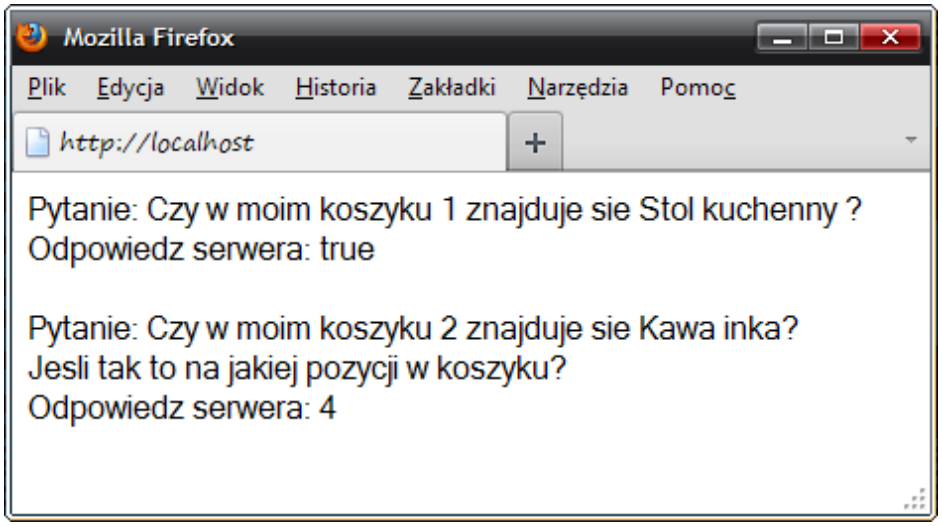
Przesuwamy wskaźnik tablicy o 3 kroki w prawo...

Wewnętrzny wskaźnik tablicy znajduje się nad elementem Stół kuchenny

Przesuwamy wskaźnik tablicy o 2 kroki w lewo...

Wewnętrzny wskaźnik tablicy znajduje się nad elementem Książka kucharska

Ćwiczenie 4.6



Ćwiczenie 4.7

localhost

localhost/

```
Tablica $MyBasket1:
Array
(
    [0] => Telewizor
    [1] => Ksiższka kucharska
    [2] => Słuchawki
    [3] => Stół kuchenny
    [4] => Kawa inka
)

Tablica $arr:
Array
(
    [0] => Łóżko
    [1] => Ksiższka informatyczna
    [2] => Słuchawki
    [3] => Nóż kuchenny
    [4] => Kawa inka
)
```

```
Jakie elementy należš do tablicy $MyBasket i $arr:
Array
(
    [2] => Słuchawki
    [4] => Kawa inka
)

Jakimi elementami różniš się te tablice?:
Array
(
    [0] => Telewizor
    [1] => Ksiższka kucharska
    [3] => Stół kuchenny
)
```

Zadanie 1

Napisz skrypt obliczający liczbę elementów w tablicy podzielnych i niepodzielnych przez dwa.

Zadanie 2

Napisz kod, który z jednej tablicy zawierającej wartości liczbowe oraz ciągi znaków utworzy dwie oddzielne tablice – jedną zawierającą wyłącznie ciągi znaków i drugą zawierającą wyłącznie wartości liczbowe.

Zadanie 2

Napisz program, który do tablicy dwuwymiarowej o wielkości 3x3 wpisze wartości losowe z zakresu –5…5 wyświetli ją w tabeli HTM